Sunday @ 11:59pm - Proj 3 now due

Proj. 4 released tomorrow

Cache coherence

Snoopy cache coherence

Ex MSI

Cache block may be...

	My copy	Can others have it?	Memory's copy
Modified	dirty	NO	stale
Shared	clean, correct	possibly	correct
Invalid	invalid	unknown	unknown

Bus requests / Actions

A processor coherence request

GetS (block) - Get clean copy of block

GetM (block) - Get a copy with intent to modify

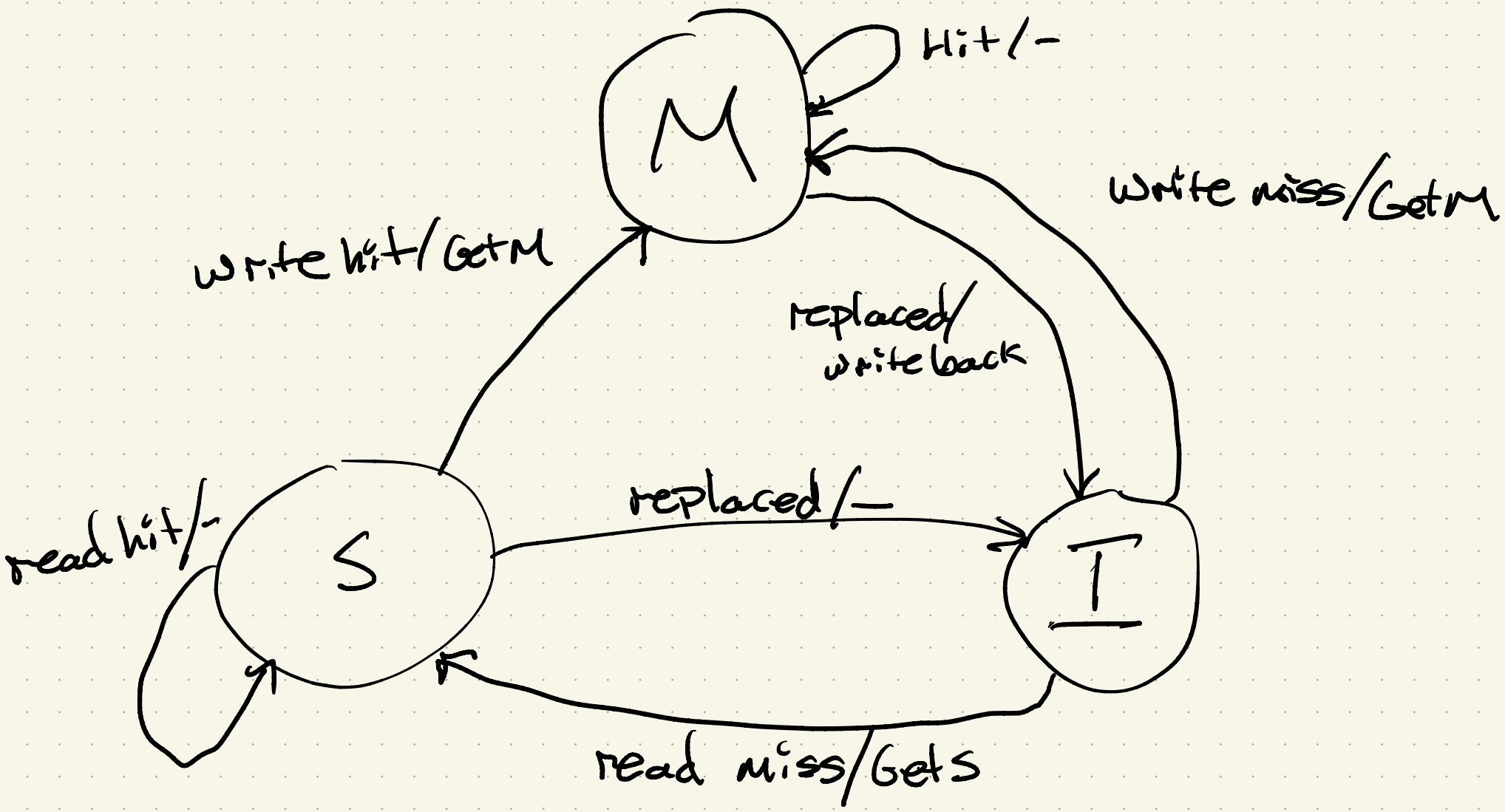
Snooper can:

Abort: stop request, retry later

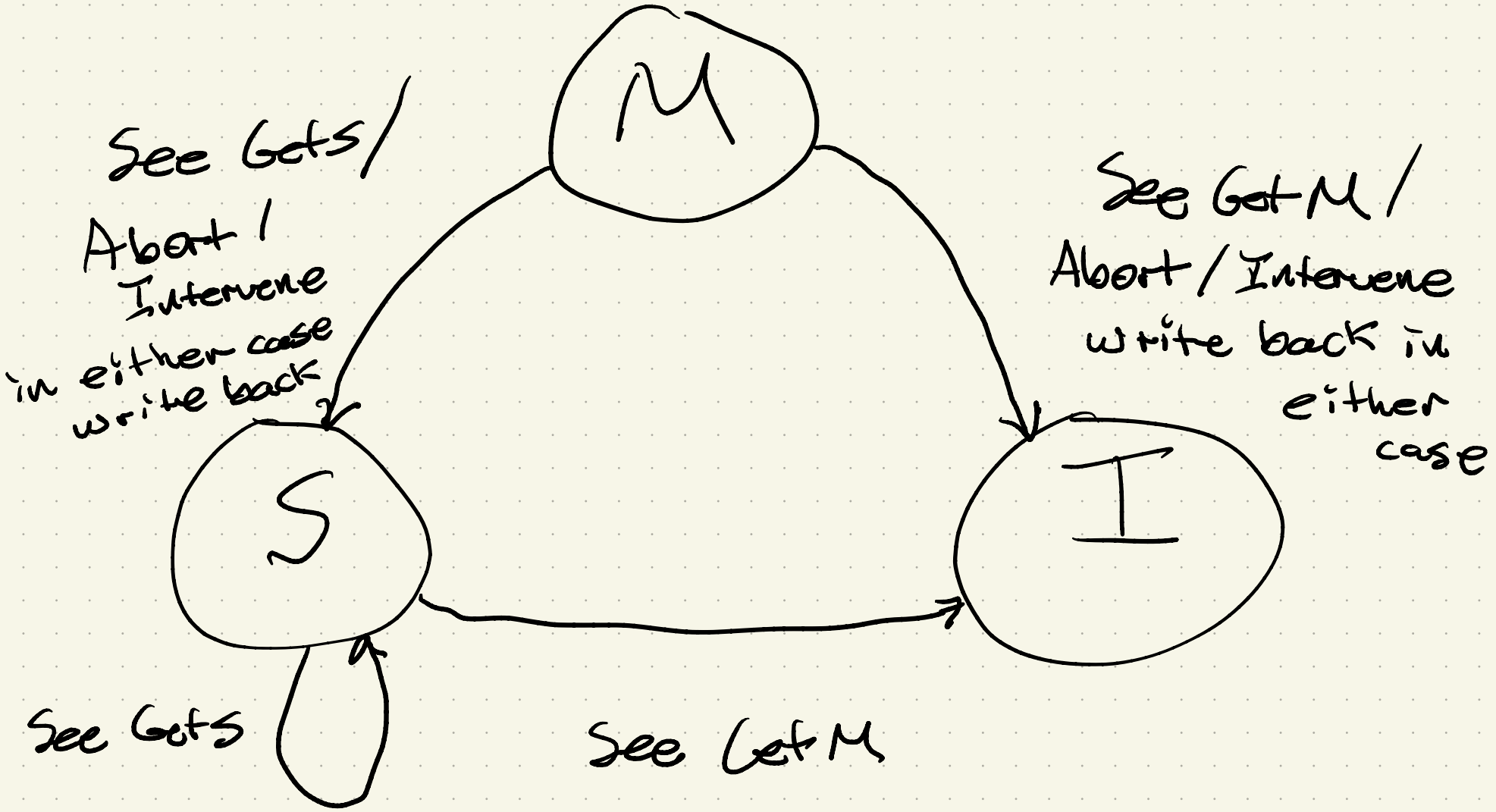
— or —

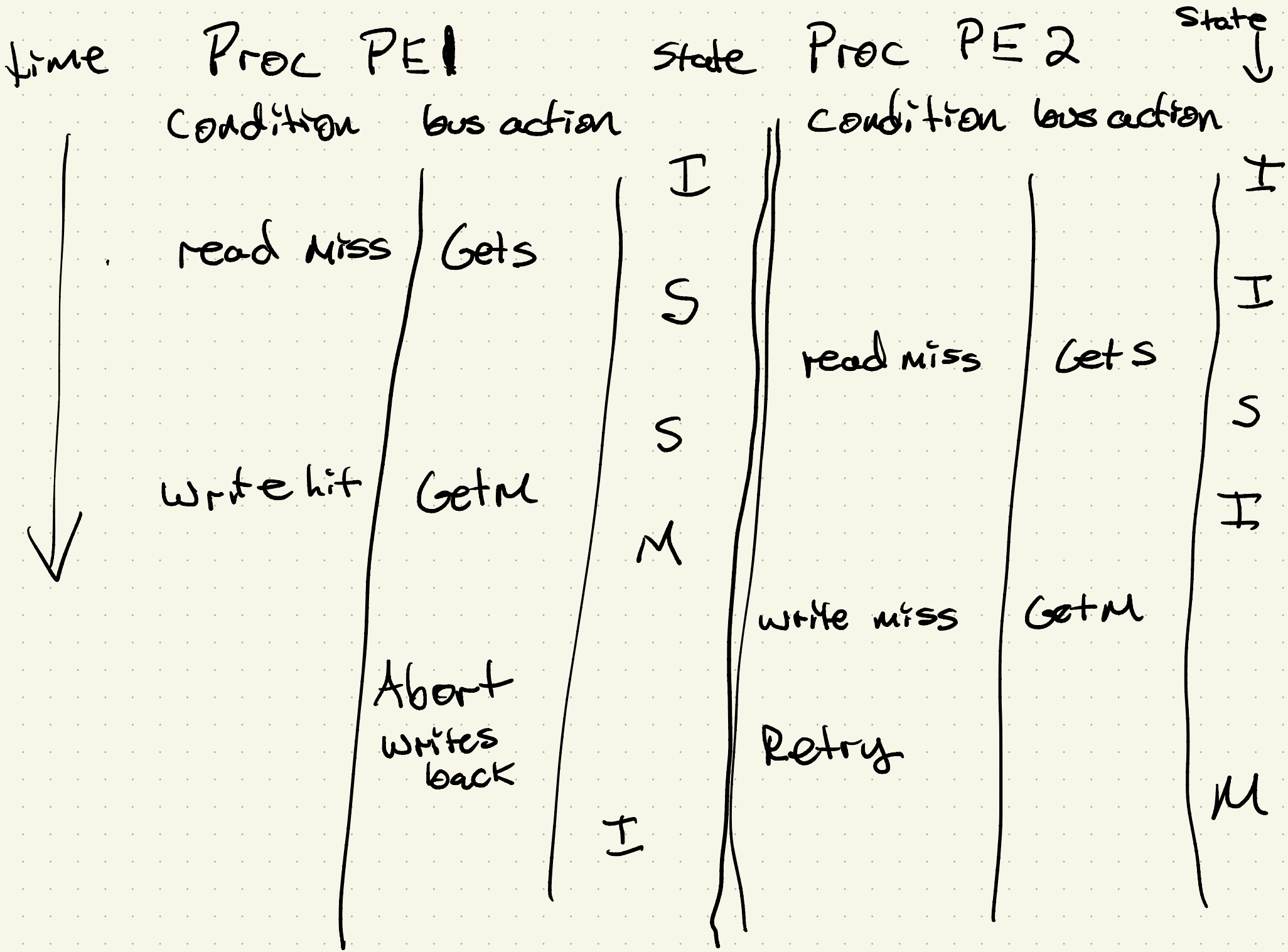
Intervene: snooper answers request in lieu of memory

Processor $'s perspective



Snooper's perspective




Time
Proc PE1
condition
bus action
read miss
Gets
write hit
GetM
Abort
writes
back
State
I
S
S
M
I
Proc PE2
condition
bus action
read miss
Gets
write miss
GetM
Retry
State
I
I
S
I
M

Abort vs. Intervene

wait on mem

cache to cache transfer

in MSI, can only perform $ to $ transfer if there's a proc in state M

why:

PE1	PE2	PE3
S	S	I, gets

Second optimization:

P1 has block in S

write hit, must broadcast GetM

"P1 has block, clean, only copy"

write hit, no need to broadcast GetM

"silent upgrade to M"

Exclusive M, E, S, I

PE1 PE2 PE3

E I I, Gets

can do $ to $ transfer

state	My copy	Can others have it?	Memory's copy	$ to $ transfer agent	WB?	Notes
M	dirty	not possible	stale	yes, 1 time	yes	
S	clean	Possibly	correct or unknown*	No	No	* if O state
E	clean	not possible	correct	yes, 1 time	No	
O owner	dirty	Possibly	stale	yes	yes	
F "Forwarder"	clean	possibly	correct	yes	No	
I	invalid	possible	unknown	No	No	

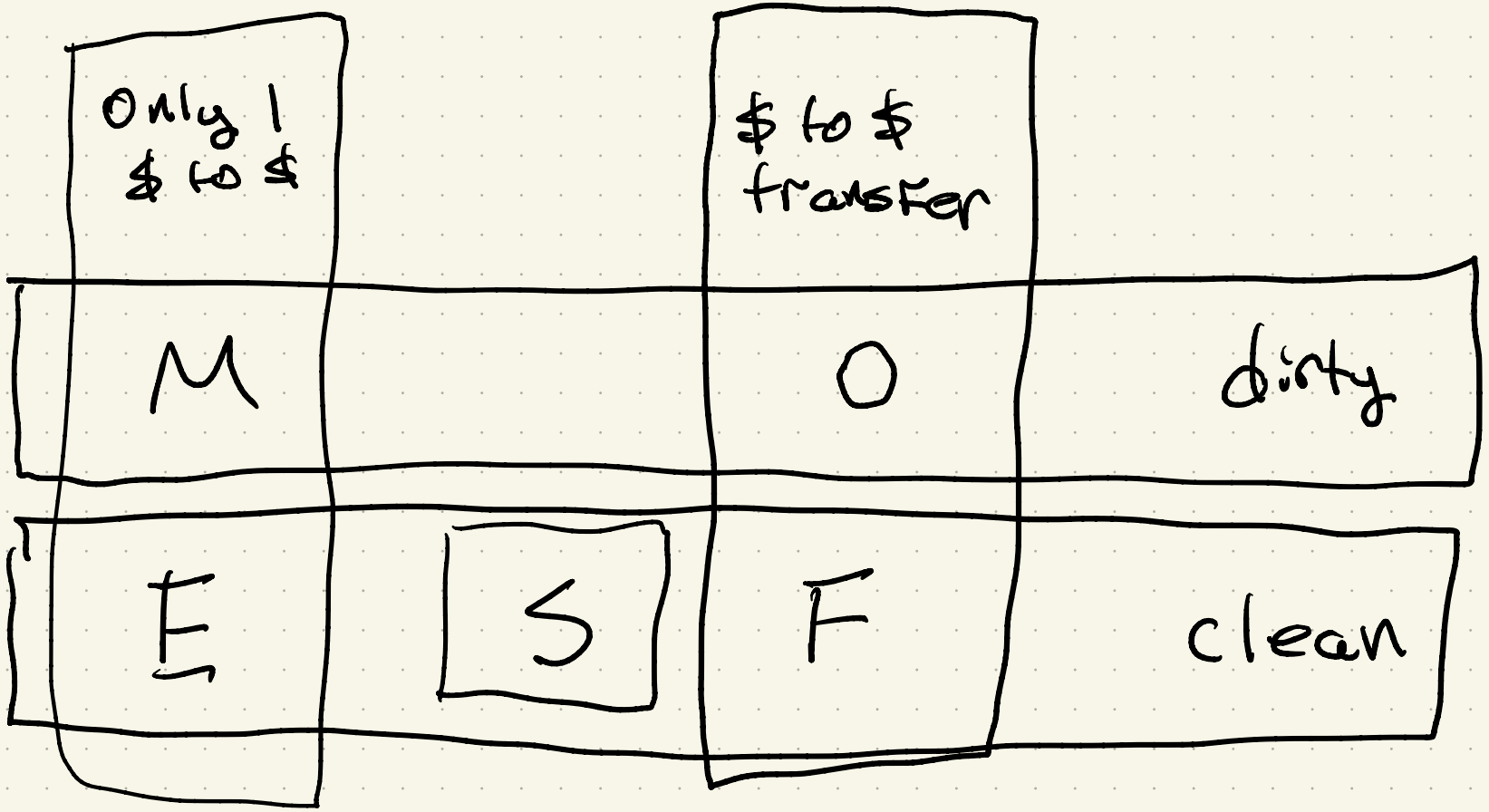
MESI

MOSIF

MOESIF

MI

SI



I

"permission"

M > E > O > F > I

O S S S

M I I I

States != protocols

MOESIF

False sharing Function of block size

```
int i   // modified a lot
int j   // read only
```

i's block M, S, I ...

j's block S

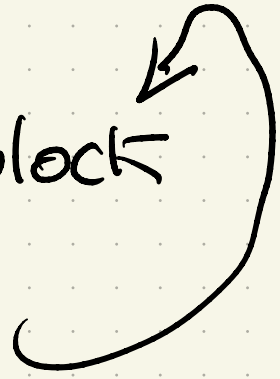
i and j in same block - can cause lower performance

"Fourth C": Coherence miss for j

Kinds of sharing patterns

Read-only: (constant)

Producer/consumer: PE x writes to block
PE y reads block



Migratory: modified block that moves between many PEs (eg, var in critical section)

write-shared: migratory but mostly written to once, ie index of loop

Coherence write policies

Write-invalidate (what we've been talking about)

Write to shared data forces sharers to invalidate

Write-update

Write to shared data causes update of sharers

Harder to verify correctness

Far less common